Operating systems

2. What are the two main roles of an operating system?

- Provides abstract machine for programmers
- Resource management

3. Given a high-level understanding of file systems, explain how a file system fulfills the two roles of

- Provides abstraction: file
- Coordinates access to the underlying resource

1. What are some of the differences between a processor running in privileged mode (also called kernel mode)

- kernel mode → all instructions
  → all memory accessible
- user mode → subset of kernel-mode

4. Which of the following instructions (or instruction sequences) should only be allowed in kernel mode?

a. Disable all interrupts

b. Read the time of day clock

c. Set the time of day clock

d. Change the memory map

e. Write to the hard disk controller register

f. Trigger the write of all buffered blocks associated with a file back to

a, c, d and e should only be performed in Kernel mode

System call interface

10. On Unix, which of the following are considered system calls? Why?

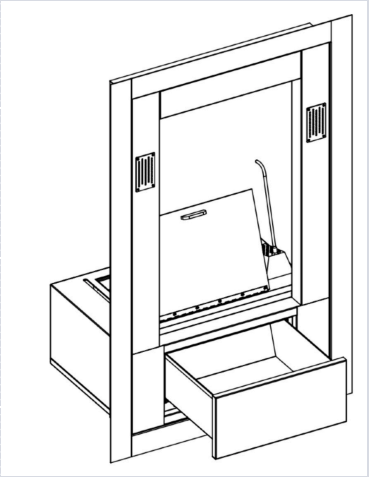
a. read() syscall

b. printf() hybrid

c. memcpy() library function

d. open() syscall

e. strncpy() library function



(night window)

5. The code in `5.c` contains the use of typical Unix process management system calls: fork(), execl(),

a. What is the value of i in the parent and child after fork?

i is the same in both

b. What is the value of my_pid in the parent after a child updates it?

unchanged

c. What is the process id of /bin/echo?

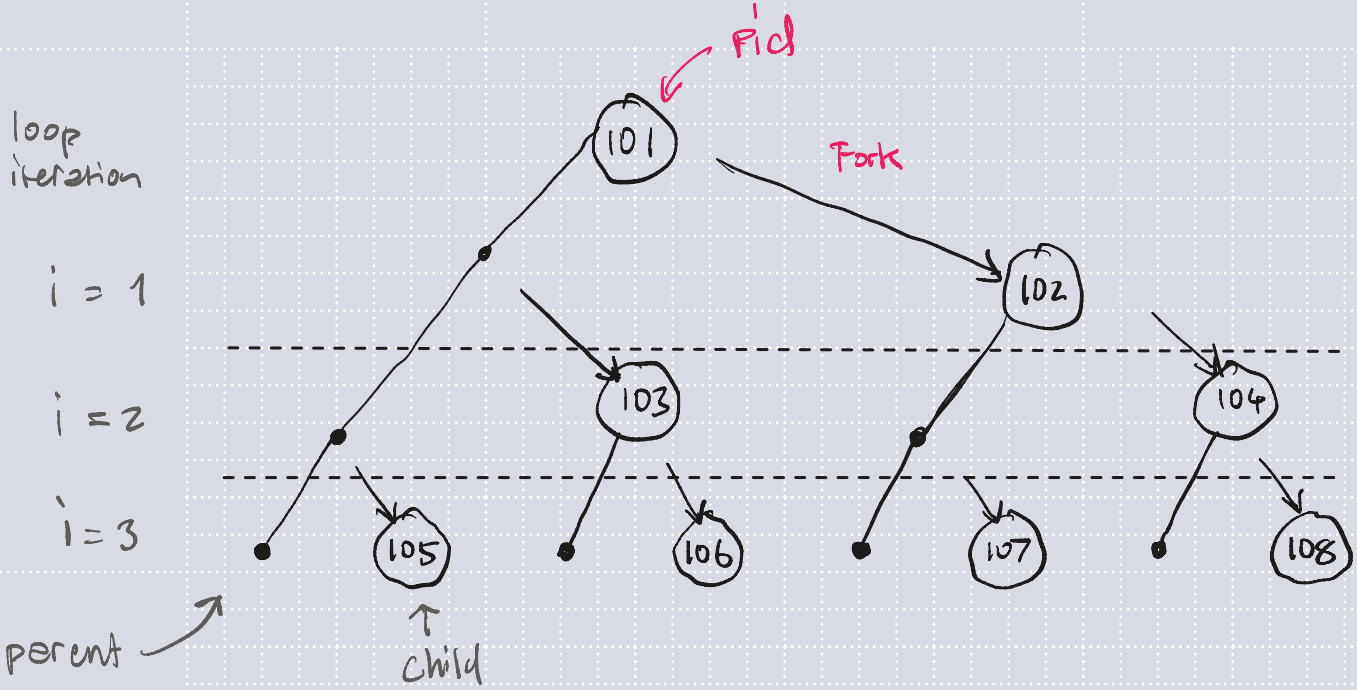
unchanged

d. Why is the code after execl not expected to be reached in the normal case? we begin executing a new program.

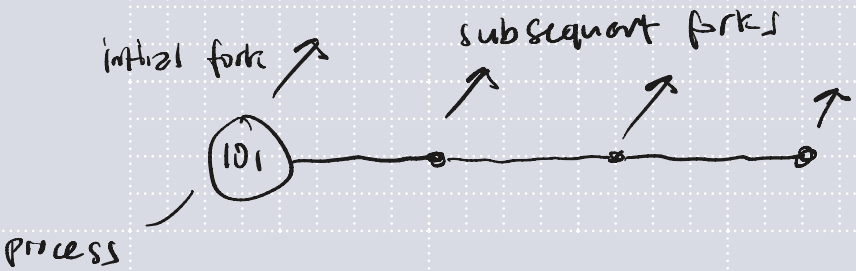
e. How many times is 'Hello World' printed when FORK_DEPTH is 3? 4

f. How many processes are created when running the code (including the first process)? 8

see diagram



i = FORK-DEPTH ; we echo in the child



6. a. What does the code in `6.c` do?

- Writes a string to a file, and creates it if it does not exist.

b. In addition to O_WRONLY, what are the other 2 ways one can open a file?

See `man 2 open`!

- O_RDONLY
- O_RDRW

c. What does open return in fd? What is it used for?

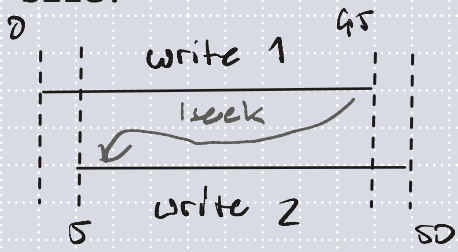
- A file descriptor (an int)
  - If >0, its a handle that can be used in other file syscalls.
  - -1 on error.

7. The code in `7.c` is a variation of the previous code that writes twice.

a. How big is the file (in bytes) after the two writes?

50 bytes

b. What is lseek() doing that is affecting the final size?



`lseek` moves the file pointer; the OS keeps track of where in the file we are

c. What other options are there in addition to SEEK_SET?

See `man 2 lseek`: SEEK_CUR, SEEK_END

8. Compile either of the previous two code fragments on a UNIX/Linux machine and run strace ./a.out and observe the output.

a. What is strace doing?

Shows which syscalls are executed during process execution.

b. Without modifying the above code to print fd, what is the value of the file descriptor used to write to the open file?

fd = 3 (seen in openat, write, close syscalls)

c. printf does not appear in the system call trace. What is appearing in its place? What's happening

`printf` sets up a buffer according to the format string; the buffer is then written to the console via a `write` syscall

9. Compile the code in `9.c`.

a. What does the following code do?

- Changes the working directory of the process to its parent directory then executes `ls`.

b. After the program runs, the current working directory of the shell is the same. Why?

- The shell forks before executing chdir, so only the child process is effected.

c. In what directory does /bin/ls run in? Why?

Where
- The working directory of the child

Why
- `exec` only changes which program runs, not the inherited environment.

Processes and threads

11. In the three-state process model, what do each of the three states signify? What transitions are possible between each of the states, and what causes a process (or thread) to undertake such a transition?

See slides 16-17 of 'Processes and Threads'.

Additionally,

- Running → Ready (higher priority process ready)
- Ready → Running (scheduler chose this process to run)
- Blocked → Ready (resource now available)

12. Given N threads in a uniprocessor system. How many threads can be running at the same point in time? How many threads can be ready at the same time? How many threads can be blocked at the same time?

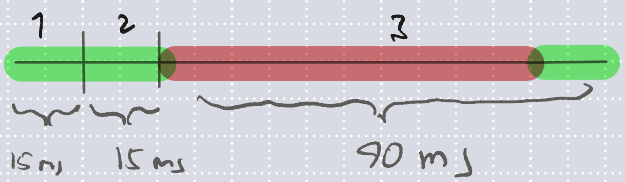
Running: 0 or 1

Blocked: N - Running - Ready

Ready: N - Running - Blocked

13. Compare reading a file using a single-threaded file server and a multi-threaded file server. Within the file server, it takes 15 msec to get a request for work and do all the necessary processing, assuming the required block is in the main memory disk block cache. A disk operation is required for one third of requests, which takes an additional 75 msec during which the thread sleeps. How many requests/sec can a server handle if it is singled threaded? If it is multithreaded?

Single-threaded:



$\frac{2}{3} \times 15\,ms + \frac{1}{3} \times 90\,ms$

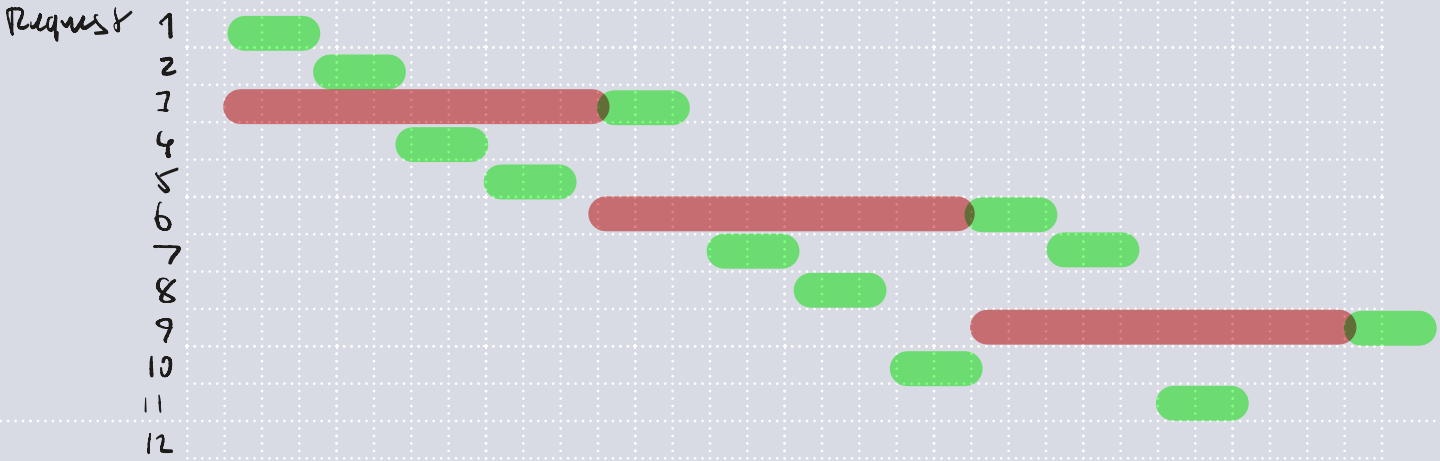
$= 40\ ms$

$\frac{1000\,ms}{40\,ms} = 25$ requests/s

running

waiting for disk I/O

Multi-threaded:



While blocked on cache miss, we can always action a cache hit.

Effectively 15 ms per request, while receiving more requests.

$\frac{1000\ ms}{15\ ms} = 66\frac{2}{3}$ request/s

Critical sections

14. The following fragment of code is a single line of C code. How might a race condition occur if it's executed concurrently in multiple threads? Can you give an example of how an incorrect result can be computed for x?

```
x = x + 1;
```

(assume x is global)

↑ Statement is NOT atomic

Thread 1:

lh t1 x

addi t1 1

sh t1 x

Thread 2:

lh t1 x

addi t1 1

sh t1 x

context switches

Thread 1 has a stale copy of x, so x is only incremented by 1, rather than 2.

15. The following function is called by multiple threads (potentially concurrently) in a multi-threaded program. Identify the critical section(s) that require(s) mutual exclusion. Describe the race condition or why no race condition exists.

(Deferred to Week 3.)

```
int i;

void foo()
{
    int j;

    /* random stuff */

    i = i + 1;
    j = j + 1;

    /* more random stuff */
```

16. The following function is called by threads in a multi-thread program. Under what conditions would it form a critical section.

```
void inc_mem(int *iptr)
{
    *iptr = *iptr + 1;
```

(Deferred to Week 7.)